

Configurable Static Type Checking for Forth

Evaluator and Examples

Jaanus Pöial
Tallinn University of Technology

EuroForth 2026

Static Type Checking

- Detect stack errors before execution
- Detect type mismatches early
- Precisely document word behaviour

Research Evolution

- 1990 - 1998: Stack Effect Calculus
- 2002 - 2006: Typed Wildcards and Inheritance
- 2008: Typing Tools for Typeless Stack Languages (Java prototype)
- 2026: Configurable Static Type Checking for Forth – Type checker in gforth

From an informal comment to a checkable contract

```
: SQUARED ( n -- n2 ) DUP * ;
```

INFORMAL SEMANTIC COMMENT

n^2 says what the result means mathematically.

- Useful documentation
- May be absent or stale
- Consumed as source text; not trusted as an assertion

CHECK

TYPED SYMBOLIC EFFECTS

```
DUP ( X[1] -- X[1] X[1] )  
*   ( n n -- n )
```

Inferred:

```
SQUARED ( n -- n )
```

The checker verifies stack balance, cell types and cell identity, not the mathematical claim that the result equals n^2 .

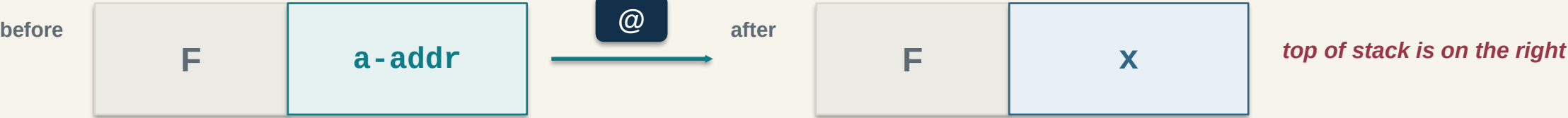
Stack effect

required input suffix

@ (a - addr - - x)

guaranteed output suffix

Any deeper frame F passes through unchanged



An effect is a typed symbolic pre/post contract over only the data-stack suffix (top right) a phrase inspects.

A boundary can refine or fail

COMPATIBLE

DP **C@** **0=**

DP (-- a-addr)

C@ (c-addr -- char)

0= (n -- flag)

result: (-- flag)

The trace retains a-addr because it is more specific than c-addr.

CLASH

C@ **!**

C@ (c-addr -- char)

! (X a-addr --)

char

produced

×

a-addr

required on top

The checker rejects the incompatible boundary before the source can attempt the bad store.

Types express the project's compatibility policy

CONVENIENT FORTH-LIKE PROFILE

more specific → more general

a-addr < c-addr < addr < X

char < n < X

flag < n < X

Useful when the target treats flags and characters as numeric cells.

STRICTER STANDARD-LIKE PROFILE

the selected file makes these choices explicit

a-addr < c-addr < addr < u < n < x

char < +n < u < n < x

flag < x (separate from n)

Aliases may name one canonical type: X, x, and cell can be equivalent.

Composition keeps the more specific comparable boundary type; incomparable types clash.

Indices remember which cell is which

DUP

(X[1] -- X[1] X[1])

one input, two correlated
outputs

SWAP

(X[2] X[1] -
X[1] X[2])

two origins exchange
positions

OVER

(X[2] X[1] -
X[2] X[1] X[2])

the deeper input reappears on
top

same index = same abstract origin • **different indices** do not claim unequal concrete values • **no index** = fresh symbolic item

Indices are local logical variables not addresses or runtime tags. The checker freshens them for every word occurrence.

Three operations used for control structures

SEQUENCE

$$p \cdot q$$

Boundary composition

Can q consume what p leaves?
Match from the top inward and propagate the more precise type through every correlated occurrence.

$$\begin{aligned}(a \dashv b) \cdot (eb \dashv d) &= (ea \dashv d) \\ (a \dashv fc) \cdot (c \dashv d) &= (a \dashv fd)\end{aligned}$$

ALTERNATIVES

$$p \sqcap q$$

Common effect

Attempt to reconcile both path effects into one interface. This is a single must-style syntactic summary—not a set of every possible behavior.

$$(a \dashv b) \sqcap (ea \dashv eb) = (ea \dashv eb)$$

$$\begin{aligned}(ea \dashv eb) \sqcap (ec \dashv ed) &= \\ (e \min(a, c) \dashv e \max(b, d))\end{aligned}$$

REPETITION

$$p \sqcap (p \cdot p)$$

One-versus-two

Compare one pass with two consecutive passes. Accept only when the implemented merge finds a locally stable summary.

The untyped base has a polycyclic-monoid structure; the checker adds partial, subtype-aware, identity-sensitive matching.

Loops receive a local stability check not an invariant proof

```
: ULOOP BEGIN DUP 0= UNTIL ;
```

IMPLEMENTED ONE-VERSUS-TWO TEST

$p = \text{DUP } 0= \text{UNTIL}$

one pass
 p

two passes
 $p \cdot p$

□

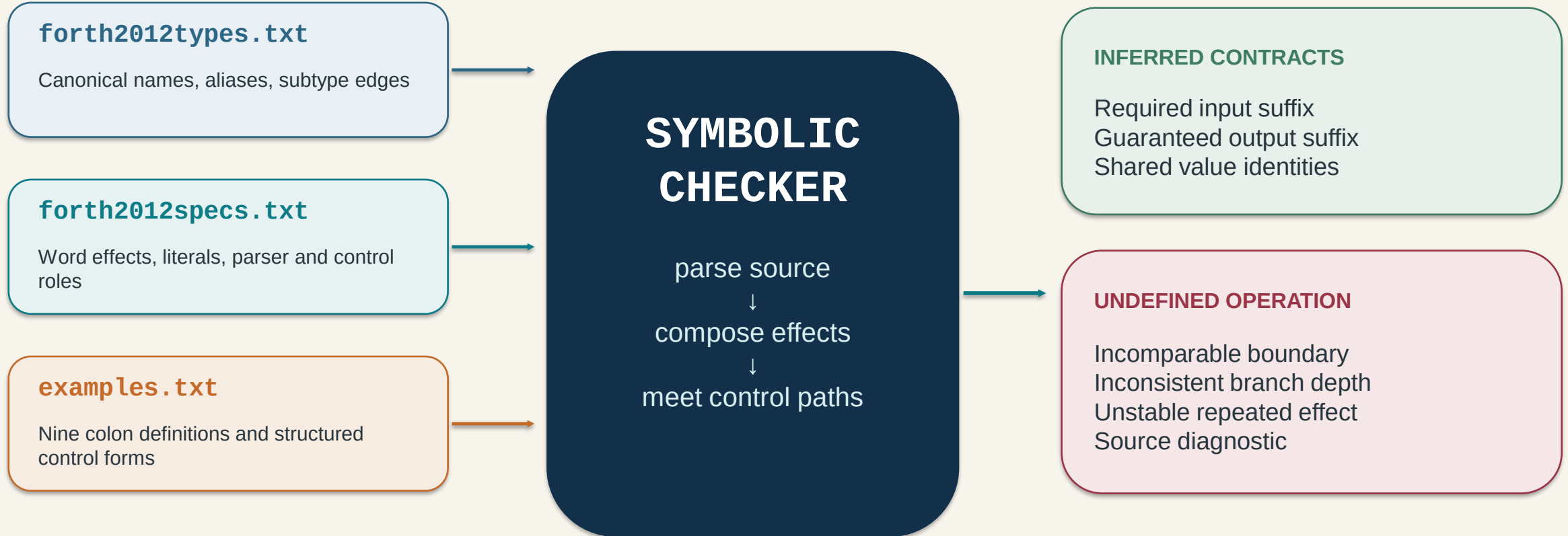
`ULOOP ( n[1] -- n[1] )`

WHAT THIS RESULT SAYS

- **Useful:** detects many bodies that grow, shrink, or change incompatibly on a second pass.
- **Local only:** the duplicated copy feeds `0=` and `UNTIL`; the original `n` remains.
- **Not proved:** termination, every iteration count, or a general inductive loop invariant.

Treat the reported loop effect as a finite stack-stability approximation, not as full loop verification.

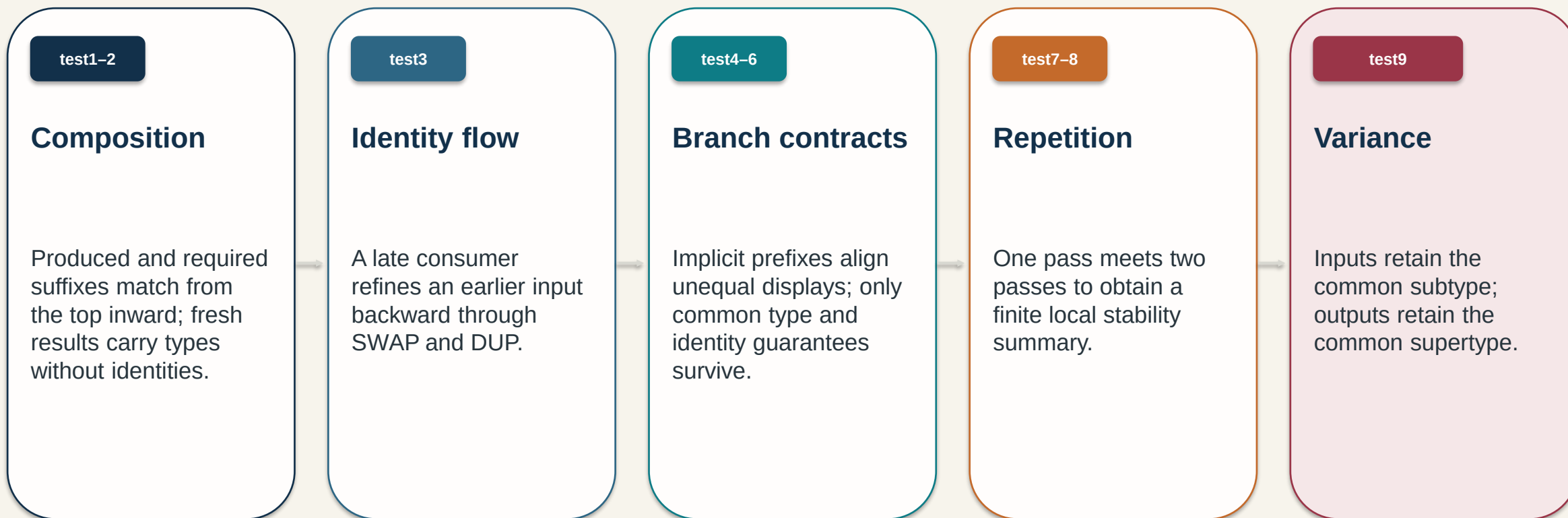
Three inputs produce one inferred dictionary



No concrete values are executed. At ; the inferred body effect becomes the specification of the new word.

Concepts covered

The examples are deliberately small: each makes one part of the effect calculus visible.



```
: test1 ( -- value ) 3 4 5 - + ;
```

: test1	empty	
3 (-- n)	n	3
4 (-- n)	n n	3 4
5 (-- n)	n n n	3 4 5
- (n n -- n)	n n	3 -1
+ (n n -- n)	n	2
;	Results in (-- n)	

```
: test2 ( fl1 fl2 -- FALSE flor )  
  OR FALSE SWAP ;
```

```
: test2  
OR ( flag flag -- flag )  
FALSE ( -- flag )  
SWAP ( X[2] X[1] – X[1] X[2] )  
;
```

flag flag

flag

flag flag

flag flag

Results in (flag flag -- flag flag)

```
: test3 ( addr X -- X addr value )  
  SWAP DUP @ ;
```

: test3	a-addr[2] X[1]
SWAP (X[2] X[1] -- X[1] X[2])	X[1] a-addr[2]
DUP (X[1] -- X[1] X[1])	X[1] a-addr[2] a-addr[2]
@ (a-addr -- X)	X[1] a-addr[2] X
;	Results in
	(a-addr[2] X[1] -- X[1] a-addr[2] X)

```
: test4 ( n incr -- n1 )  
  IF 1 + FI ;
```

: test4	n flag
IF (flag --)	n
1 (-- n)	n n
+ (n n -- n)	n
FI	Branch effect (n -- n)
;	Results in (n flag -- n)

If-branch body effect (n – n) is more exact than empty branch effect (--)


```
: test5 ( X1 flag -- X1 X2 )  
  IF DUP ELSE 0 FI ;
```

: test5	X[1] flag
IF (flag --)	X[1]
DUP (X[1] -- X[1] X[1])	X[1] X[1]
ELSE	If branch effect (X1 -- X1 X1)
0 (-- n)	X[1] n
FI	Else-branch effect (-- n)
;	Meet: (X1 -- X1 X2)

Prefer supertype in output (we don't know which branch was chosen and n is subtype of X)
Results in (X[1] flag -- X[1] X)

```
: test6 ( X2 X1 ad fl -- X3 X4 X5 )  
  IF ROT ELSE @ FI ;
```

: test6	X2 X1 a-addr flag
IF (flag --)	X2 X1 a-addr
ROT (X[3] X[2] X[1] -- X[2] X[1] X[3])	X1 a-addr X2
ELSE	If (X2 X1 a-addr -- X1 a-addr X2)
@ (a-addr -- X)	X2 X1 X3
FI	Else (a-addr -- X)
;	Meet: (X2 X1 a-addr -- X3 X4 X5)

Prefer supertype in output (we don't know which branch was chosen, a-addr is subtype of X)
Results in (x x a-addr flag -- x x x)

```
: test9 ( ad fl -- X )  
  IF @ ELSE C@ FI ;
```

: test9	a-addr flag
IF (flag --)	a-addr
@ (a-addr -- X)	X
ELSE	If (a-addr -- X)
C@ (c-addr -- char)	char
FI	Else (c-addr -- char)
;	Meet: (a-addr -- X)

Prefer subtype in input (a-addr is subtype of c-addr and X is supertype of char)
Results in (a-addr flag -- X)

```
: test7 ( fl2 fl1 -- fl3 fl4 )  
  BEGIN SWAP OVER WHILE INVERT REPEAT ;
```

: test7	fl2 fl1
BEGIN	Loop1 effect: (X X -- X X)
SWAP (X[2] X[1] -- X[1] X[2])	fl1 fl2
OVER (X[2] X[1] -- X[2] X[1] X[2])	fl1 fl2 fl1
WHILE	fl1 fl2
INVERT (flag -- flag)	fl1 fl3
REPEAT	Loop2 effect: (flag -- flag)
;	

Prefer subtype in input (flag is subtype of X)
Results in (flag flag -- flag flag)

```
: test8 ( -- )  
  BEGIN KEY 0= UNTIL ;
```

```
: test8  
BEGIN  
KEY ( -- char )      char  
0= ( n -- flag )     flag  
UNTIL ( flag -- )    Loop body ( -- )  
;
```

Prefer subtype in input (char is subtype of n)
Results in (--)

Branches and loops

```
: P INVERT HERE ! DP ! DP @ DP C@ SWAP ;
```

```
: Q SWAP ! DP @ DP ;
```

```
: test10 IF P ELSE Q FI ;
```

```
: BODY SWAP DUP @ 0= ;
```

```
: test11 BEGIN BODY UNTIL ;
```

```
: BODY23 ROT INVERT DROP 0 ;
```

```
: test12 BEGIN BODY23 KEY? UNTIL ;
```

```
BODY23 BODY23 BODY23
```

```
P ( x flag -- char x )
```

```
Q ( a-addr x -- x a-addr )
```

```
test10 ( a-addr flag flag -- x x )
```

```
BODY ( a-addr[2] x[1] -- x[1] a-addr[2] flag )
```

```
test11 ( a-addr a-addr -- a-addr a-addr )
```

```
BODY23 ( flag x[2] x[1] -- x[2] x[1] n )
```

```
test12 ( flag flag x -- x x n )
```

```
Program: BODY23 BODY23 BODY23
```

```
> flag flag flag
```

```
    BODY23    ( flag flag flag -- flag flag n )
```

```
    BODY23    ( flag flag n -- flag n n )
```

```
    BODY23    ( flag n n -- n n n )
```

```
< n n n
```

What acceptance means and what it does not

ACCEPTANCE SUPPORTS

- Declared word effects compose without a modeled clash
- Every checked branch receives one implemented common effect
- Every checked loop passes the local one-versus-two test
- Source structure, parser roles, and compilation state fit the active profile

Useful as executable stack-effect documentation.

ACCEPTANCE DOES NOT PROVE

- That trusted primitive and parser declarations match runtime behavior
- A path-sensitive set of every branch behavior
- Termination or an inductive invariant for arbitrary loop counts
- Unrestricted EXECUTE, return-stack tricks, non-local THROW, or metaprogramming

Use alongside tests, review, and target-system knowledge.

Known limits include lossy identity compaction, one effect per primitive, and the one-versus-two loop approximation.

Summary

types + specifications + source → inferred specifications / alarms

1

Composition is sequential

A late boundary refinement propagates through every occurrence of the same symbolic value. Type information flows back and forth in a linear sequence of words.

2

Branch meet is conservative

Inputs retain the common subtype; outputs retain the common supertype; identities must hold on every path.

3

Repetition is deliberately finite

One-versus-two gives a useful local summary—not termination or a general loop invariant.